

VEERMATA JIJABAI TECHNOLOGICAL INSTITUTE (VJTI)

Mumbai – 400 019

Department of Computer Engineering

SYSTEM DESIGN DOCUMENT

Food Donation and Lifecycle Management System

Software Engineering Laboratory – Experiment 8A

Student Name	Roll Number
Rudrakshi Chincholkar	241071014
Nihira Neralwar	241071049

Academic Year: 2025–26

April 2026

SE Lab 8A | Group C

Table of Contents

1 Design Goals

Design goals articulate the non-functional quality criteria that govern every architectural and implementation decision in the Food Donation and Lifecycle Management System (FDLMS). They are derived from stakeholder requirements established during the requirements phase (Experiments 3A and 4B) and are prioritised as Critical, High, or Medium.

1.1 Performance

Goal ID	Priority	Metric / Constraint
DG-P1	Critical	All donation browse, dashboard, and listing pages must load within 3 seconds under a concurrent load of 500 users.
DG-P2	Critical	Lifecycle state transitions (e.g., Submitted → Inspected → Accepted) must be persisted and reflected in the UI within 2 seconds of the triggering action.
DG-P3	High	Real-time notifications must be dispatched to all relevant stakeholders within 3 seconds of any lifecycle event.
DG-P4	High	Donation-matching engine queries must return ranked results within 1 second for a dataset of up to 10,000 active donations.
DG-P5	Medium	Audit log and report generation must complete within 10 seconds for up to 50,000 records.

1.2 Reliability and Availability

Goal ID	Priority	Metric / Constraint
DG-R1	Critical	The system must achieve 99.5% uptime during active platform operation, excluding scheduled maintenance windows of no more than 2 hours per week.
DG-R2	Critical	All donation lifecycle transitions, inspection outcomes, and delivery confirmations must be persisted with zero data loss via ACID-compliant transactional writes.
DG-R3	High	The system must recover from a single-node failure within 30 seconds using health-check-based failover.
DG-R4	High	Failed notifications must be retried up to 3 times with exponential back-off; persistent failures are escalated to the System Administrator.

1.3 Security and Privacy

Goal ID	Priority	Metric / Constraint
DG-S1	Critical	All passwords must be stored as salted SHA-256 hashes; bcrypt (cost factor ≥ 12) is the minimum acceptable alternative.
DG-S2	Critical	Role-Based Access Control (RBAC) must enforce that no actor can access data or operations outside their permitted scope (see Section 5).
DG-S3	High	All data in transit must be encrypted using TLS 1.3; all data at rest must be AES-256 encrypted.
DG-S4	High	JWT tokens must use RS256 signing; web tokens expire after 1 hour, mobile tokens after 8 hours; refresh tokens are server-side revocable.
DG-S5	High	Accounts are locked for 30 minutes after 5 consecutive failed login attempts; each attempt is logged with timestamp and IP address.

DG-S6	Medium	Personally Identifiable Information (PII) of all actors must comply with applicable data-privacy regulations; PII must not appear in log files.
-------	--------	---

1.4 Scalability and Maintainability

Goal ID	Priority	Metric / Constraint
DG-M1	High	The system must be decomposed into loosely coupled subsystems so that any single subsystem can be modified without cascading changes to others.
DG-M2	High	New food categories, safety rule types, and notification channels must be integrable without altering existing controller interfaces (Open/Closed Principle).
DG-M3	High	The application tier must support horizontal scaling with no architectural change; the system must handle ≥ 50 concurrent users without performance degradation.
DG-M4	Medium	The codebase must maintain $\geq 70\%$ unit-test coverage for all controller and service classes.
DG-M5	Medium	All lifecycle and activity logs must be tamper-resistant, timestamped, and permanently retained for regulatory audit.

1.5 Usability

Goal ID	Priority	Metric / Constraint
DG-U1	High	Each actor's boundary interface must expose only the features relevant to that role, reducing cognitive overhead through role-scoped dashboards.
DG-U2	High	In-app and email/SMS notifications must be delivered to relevant stakeholders within 3 seconds of any donation lifecycle event.
DG-U3	Medium	The mobile-responsive UI must be fully usable on screens as small as 375 px width with no horizontal scrolling.
DG-U4	Medium	Donors must be able to submit a food donation request in no more than 5 steps from login.

2 Subsystem Decomposition

FDLMS is decomposed into six subsystems, each encapsulating a cohesive set of responsibilities. The decomposition follows the Entity–Boundary–Control (EBC) pattern established in Experiment 3A and is consistent with the class diagrams produced in Experiment 4B. Each subsystem exhibits high cohesion and low coupling; inter-subsystem communication occurs exclusively through well-defined interfaces.

2.1 Subsystem Overview

Subsystem	Key Classes	Responsibility	Primary Actors
AuthSubsystem	User, Role, SessionToken	Registration, login, logout, JWT issuance, RBAC enforcement, account lockout.	All actors
DonationSubsystem	FoodDonation, LifecycleEntry, DonationMatchingEngine	Create/edit/cancel donations; enforce lifecycle FSM; automated NGO-matching engine.	Donor, NGO, Admin
InspectionSubsystem	InspectionReport, FoodSafetyInspector, SafetyRule, Penalty	Assign inspectors; conduct inspections; flag unsafe food; issue penalties.	NGO, Inspector, Admin
LogisticsSubsystem	PickupTask, DeliveryTask, IssueReport	Volunteer pickup task management; delivery partner delivery task management.	Volunteer, Delivery Partner, NGO
NotificationSubsystem	NotificationService, Notification, NotifConfig	Dispatch real-time in-app, email, and SMS notifications; retry queue management.	All actors (system-driven)
AdminSubsystem	AuditLog, Report, UserManagement	User management; audit log access; report generation; database maintenance; system monitoring.	System Administrator

2.2 Subsystem UML Class Diagrams

The following subsections present per-subsystem class descriptions derived from the use-case realisations in Experiment 4B. Each subsystem follows the UML Entity–Boundary–Control (EBC) notation. PlantUML source code for all diagrams is provided in Part 3 of this document. Diagram placeholders are marked below; rendered diagrams should be inserted here in the final submission.

2.2.1 AuthSubsystem

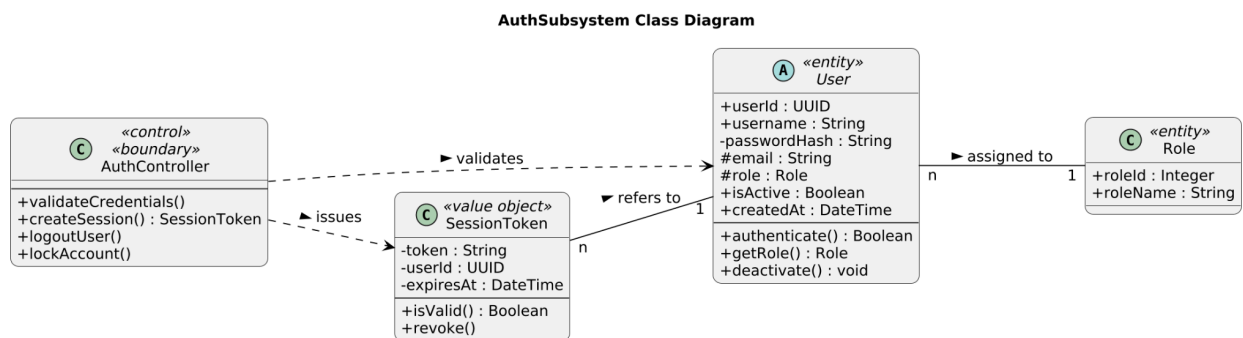


Figure 1: Academic AuthSubsystem Architecture

2.2.2 DonationSubsystem

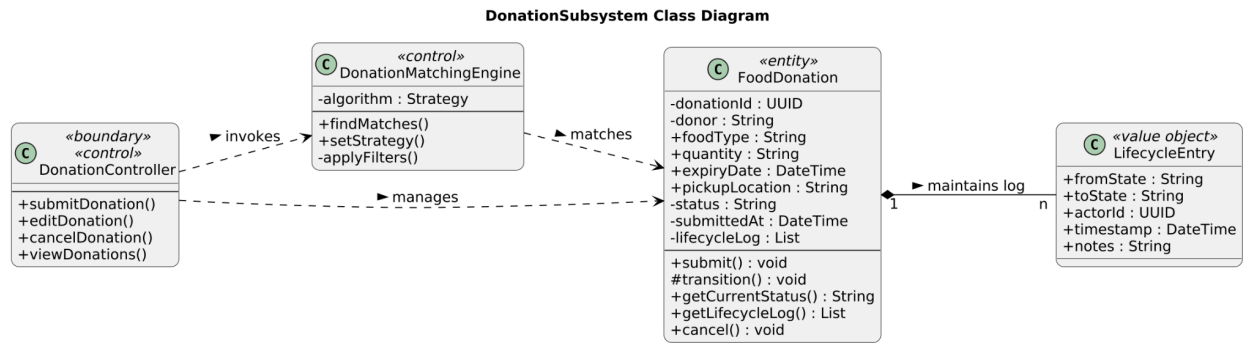


Figure 2: Academic DonationSubsystem Architecture

2.2.3 InspectionSubsystem

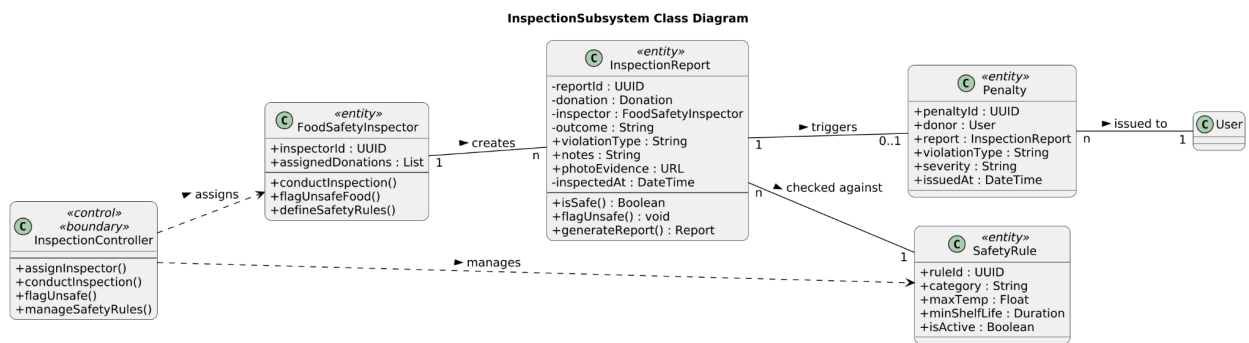


Figure 3: Academic InspectionSubsystem Architecture

2.2.4 LogisticsSubsystem

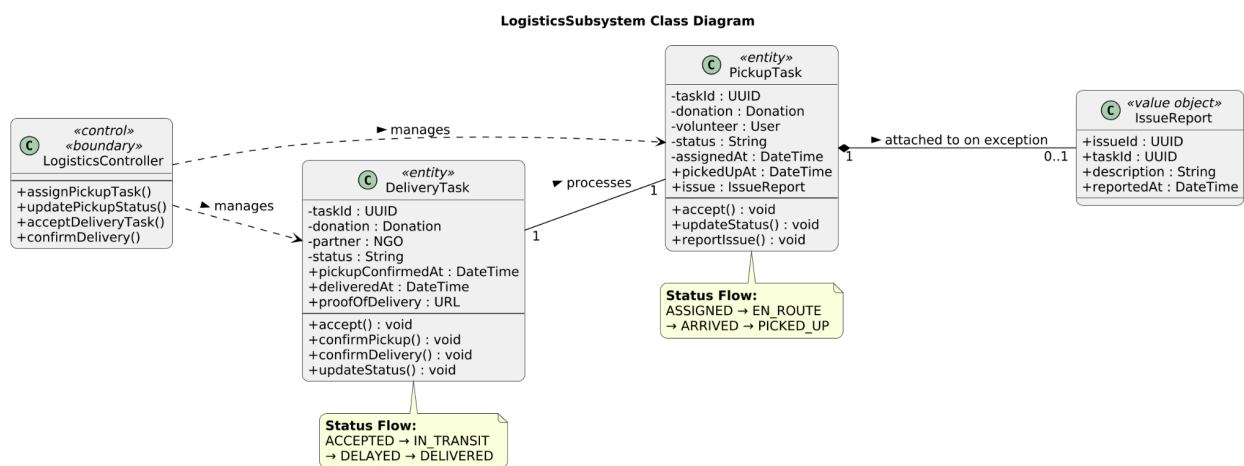


Figure 4: Academic LogisticsSubsystem Architecture

2.2.5 NotificationSubsystem

[Figure 5: NotificationSubsystem — described inline; see PlantUML Diagram 1 for full class model]

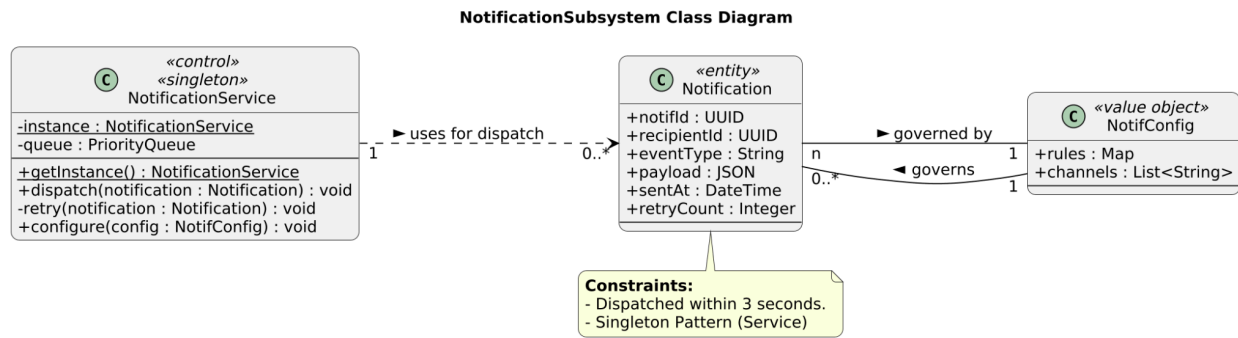


Figure 5: NotificationSubsystem Class Model

2.2.6 AdminSubsystem

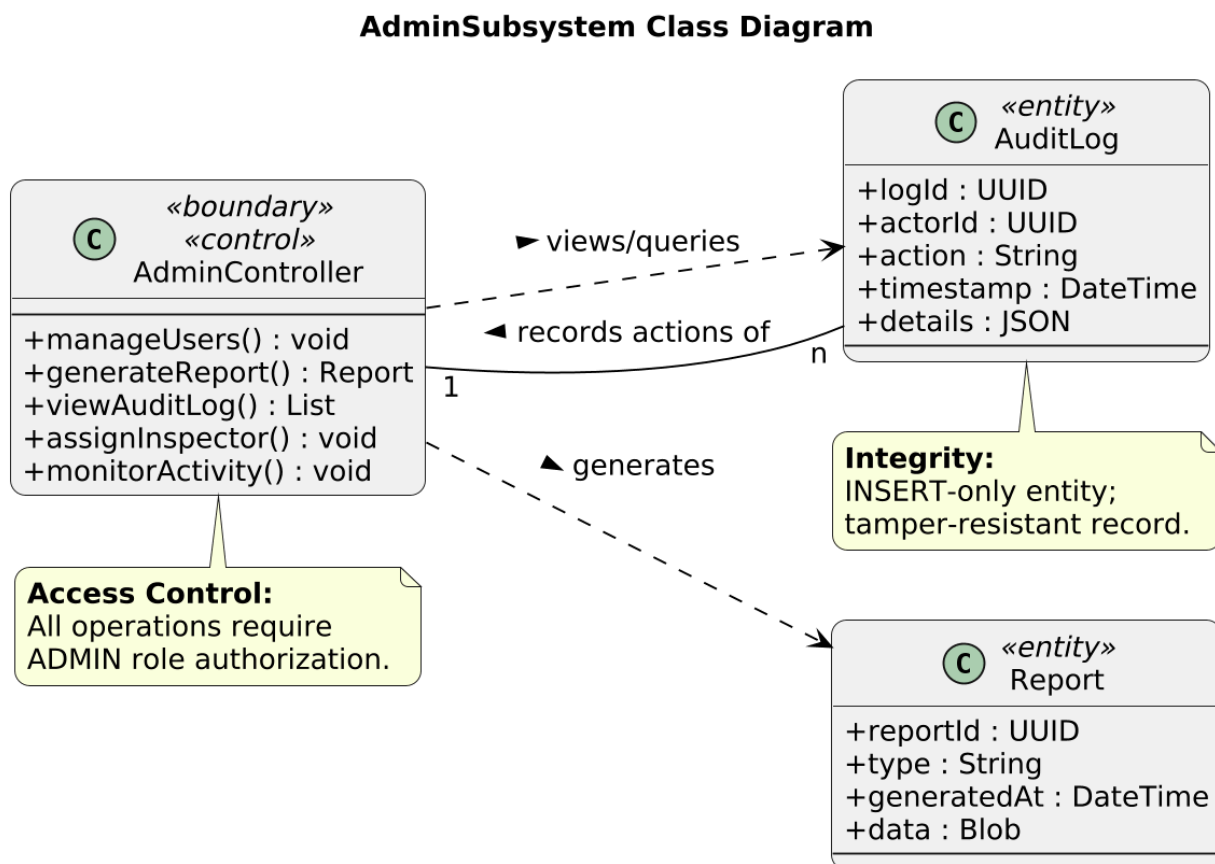


Figure 6: Academic AdminSubsystem Architecture

2.3 Full System Class Diagram

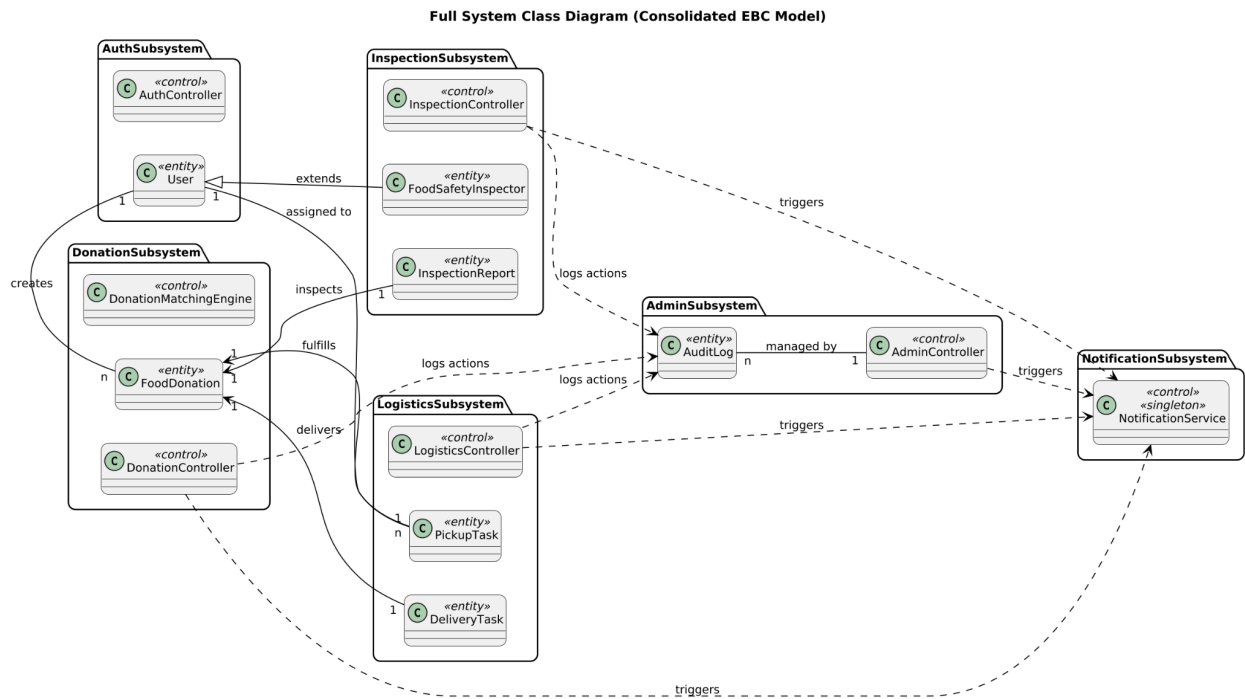


Figure 7: Consolidated System Architecture and Inter-Subsystem Dependencies

3 Hardware / Software Mapping

3.1 Deployment Architecture

FDLMS follows a three-tier client–server deployment model augmented with a dedicated asynchronous notification layer. The system is designed for deployment on commodity cloud infrastructure (AWS, GCP, or equivalent) or on-premise servers.

- 1. Presentation Tier — Browser or mobile client running the React Native / Progressive Web Application (PWA); communicates exclusively over HTTPS.
- 2. Application Tier — Node.js (or Java Spring Boot) REST API server hosting all six subsystem controllers and services; stateless; horizontally scalable behind Nginx.
- 3. Data Tier — PostgreSQL 16 (primary + read replica) for persistent ACID-compliant storage; Redis 7 for session tokens, matching result cache, and notification retry queue.
- 4. Notification Tier — Dedicated VM or serverless function hosting the WebSocket server, SMTP relay, and SMS gateway (Twilio); decoupled from the application tier.

3.2 UML Deployment Diagram

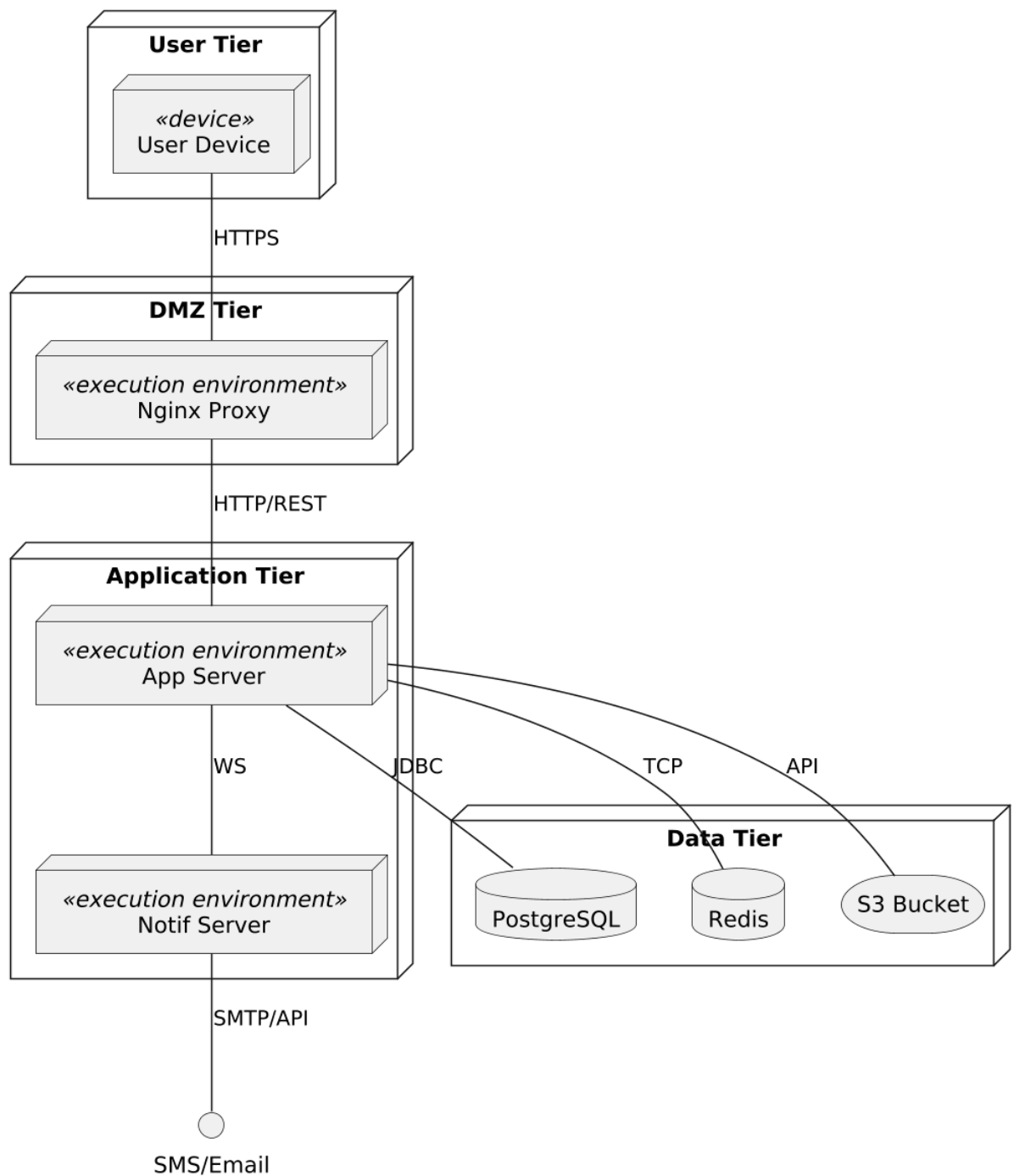
UML Deployment Diagram for FDLMS

Figure 8: Physical Architecture and Communication Protocols

3.3 Hardware Node Summary

Node	Software Artefacts	Specification
Client Device	React Native PWA (HTTPS only)	Any modern browser; Android/iOS app optional. No minimum spec enforced.

Nginx Reverse Proxy	Nginx 1.26, TLS certificates, mod_proxy, WebSocket upgrade	2 vCPU, 4 GB RAM; active-passive HA pair; SSL termination; static asset caching.
Application Server(s)	Node.js 20 LTS (or Spring Boot 3.2 / JVM 21); all six subsystem controllers	4 vCPU, 16 GB RAM per node; horizontally scalable; stateless (JWT auth).
Notification Server	WebSocket server; SMTP relay; Twilio SDK; Redis consumer	2 vCPU, 4 GB RAM; dispatches notifications ≤ 3 s; retry queue via Redis.
PostgreSQL Server	PostgreSQL 16; separate schema per subsystem; read replica	8 vCPU, 32 GB RAM, 1 TB SSD; primary + streaming read replica.
Redis Cluster	Redis 7; session store; matching cache; notification retry queue	2 vCPU, 8 GB RAM; in-memory cluster with AOF persistence.
Object Store	S3-compatible (AWS S3 / MinIO); inspection photos, delivery proof, PDFs	Managed; pay-per-use; versioning enabled for audit immutability.

4 Data Management

4.1 Persistent Data Stores

FDLMS uses PostgreSQL 16 as its single source of truth. Each entity class from the Requirements Analysis Document maps to a relational table as shown below. Schemas are namespaced per subsystem to support future microservice decomposition.

Table	Primary Key	Foreign Keys	Key Columns
users	user_id (UUID)	—	role, password_hash, is_active, created_at
food_donations	donation_id (UUID)	donor_id → users	food_type, quantity, expiry_date, status, submitted_at
lifecycle_log	log_id (UUID)	donation_id → food_donations; actor_id → users	from_state, to_state, timestamp, notes (INSERT-only)
inspection_reports	report_id (UUID)	donation_id → food_donations; inspector_id → users	outcome, violation_type, notes, photo_urls, inspected_at
safety_rules	rule_id (UUID)	author_id → users	category, max_temp, min_shelf_life, is_active
penalties	penalty_id (UUID)	donor_id → users; report_id → inspection_reports	violation_type, severity, issued_at (INSERT-only)
pickup_tasks	task_id (UUID)	donation_id → food_donations; volunteer_id → users	status, assigned_at, picked_up_at, version
delivery_tasks	task_id (UUID)	donation_id → food_donations; partner_id → users	status, proof_url, delivered_at, version
notifications	notif_id (UUID)	recipient_id → users	event_type, payload, sent_at, retry_count, status
audit_logs	log_id (UUID)	actor_id → users	action, timestamp, details (INSERT-only)

4.2 Caching Strategy

Data	Store / TTL	Invalidation Strategy
Session tokens (JWT)	Redis; TTL 1 hour (web) / 8 hours (mobile)	Refreshed on each authenticated request; invalidated on logout or deactivation.
Donation availability list	Redis; TTL 30 seconds	Invalidated on any donation status change; prevents stale listings during concurrent NGO browsing.
Matching engine results	Redis; TTL 60 seconds	Per-NGO cache keyed by criteria hash; invalidated when a matched donation changes status.
Safety rules	In-memory (application startup)	Loaded from DB on startup; refreshed on any admin CRUD operation via cache-aside.
Report aggregations	Redis; TTL 5 minutes	Computed on first request; invalidated nightly by the batch scheduler.

4.3 Data Integrity and Backup

The following integrity and backup constraints apply across all subsystems:

- **ACID Transactions:** All lifecycle transitions, inspection flag operations, and pickup/delivery confirmations are wrapped in database transactions; any failure triggers a full rollback.
- **Audit Trail Immutability:** The `lifecycle_log`, `inspection_reports`, `penalties`, and `audit_logs` tables are INSERT-only; no UPDATE or DELETE statements are permitted at the application or database level.
- **Optimistic Locking:** A version column on `food_donations`, `pickup_tasks`, and `delivery_tasks` prevents lost updates when two actors attempt to modify the same record concurrently (e.g., two NGOs accepting the same donation).
- **Soft Delete:** User accounts and donation records are soft-deleted (`is_active = false`, `is_deleted = true`); physical rows are retained for 7 years per NGO regulatory requirements.
- **Backup Schedule:** A `pg_dump` cron job runs nightly at 02:00 IST and archives compressed snapshots to an off-site S3-compatible object store; weekly full backups are retained for 3 months.
- **Data Retention:** Active donations are retained indefinitely; delivered and reconciled donations are archived to cold storage after 90 days; lifecycle logs, inspection records, and penalty records are retained permanently.

5 Access Control

5.1 RBAC Policy

FDLMS implements Role-Based Access Control enforced at every API endpoint via JWT role-claim inspection. Six roles are defined, each mapping directly to an actor identified in Experiment 3A.

Role	Description
DONOR	Registered food donor; submits, edits, and cancels donations; must follow safety guidelines; receives feedback.
NGO	Non-governmental organisation; browses and accepts donations post-inspection; manages volunteers and delivery partners.
VOLUNTEER	Community volunteer; sets availability; accepts and completes pickup tasks; reports pickup issues.
DELIVERY_PARTNER	Logistics provider; accepts, picks up, and delivers accepted donations; updates delivery status.
INSPECTOR	Food Safety Inspector; defines safety rules; conducts inspections; flags unsafe food.
ADMIN	System Administrator; full read/write on all data; manages users, assigns inspectors, generates reports, monitors activity.

5.2 Permission Matrix

Y = permitted | – = not permitted | Own = own records only | Assigned = assigned records only

Operation	Donor	NGO	Volunteer	Delivery	Inspector	Admin
Submit / Edit / Cancel Donation	Y	–	–	–	–	Y
View Donation List	–	Y	Y	–	–	Y
Accept / Reject Donation (post-inspection)	–	Y	–	–	–	–
Assign Inspector to Donation	–	Y	–	–	–	Y
Inspect Food Donation	–	–	–	–	Y	–
Flag Unsafe Food	–	–	–	–	Y	–
Define Safety Rules	–	–	–	–	Y	–
Issue Penalty to Donor	–	–	–	–	–	Y
Accept Pickup Task	–	–	Y	–	–	–
Update Pickup Status	–	–	Y	–	–	–
Report Pickup Issue	–	–	Y	–	–	–
Accept Delivery Task	–	–	–	Y	–	–
Confirm Delivery	–	–	–	Y	–	–
Track Donation Lifecycle	Y	Y	Y	Y	Y	Y
Manage Users	–	–	–	–	–	Y
Generate Reports	–	–	–	–	–	Y

Monitor System Activity	–	–	–	–	–	Y
Send Real-Time Notifications (system)	–	–	–	–	–	Y

Note: 'Track Donation Lifecycle' is scoped — Donors view own donations only; Volunteers and Delivery Partners view their assigned tasks only; Inspectors view assigned inspections only.

5.3 Authentication Flow

1. The user submits credentials (username + password) to the Login Page boundary object via HTTPS POST.
2. AuthController.validateCredentials() retrieves the stored salted SHA-256 hash from the users table and verifies the submitted password.
3. On success, AuthController.createSession() issues a signed RS256 JWT (1-hour web / 8-hour mobile) and stores the corresponding refresh token in Redis.
4. Every subsequent API request presents the JWT in the Authorization: Bearer header; the middleware validates the token signature and expiry, then extracts the role claim.
5. RBAC checks are applied per endpoint before any business logic executes; failed checks return HTTP 403 and are logged to the audit_log table.
6. On 5 consecutive failed login attempts, the account is locked for 30 minutes; the lockout event is logged with the actor's IP address.
7. On logout, the server-side refresh token is revoked in Redis; the client discards the JWT.

6 Control Flow Mechanisms

FDLMS employs three complementary control flow patterns: synchronous request–response (procedure-driven), asynchronous event-driven processing, and scheduled batch jobs.

6.1 Procedure-Driven Control (Synchronous)

The majority of FDLMS interactions follow a synchronous request–response pattern:

8. 1. The boundary object (React / PWA component) issues an HTTPS GET or POST to a REST endpoint.
9. 2. Nginx validates the TLS connection and forwards the request to the application server cluster.
10. 3. The corresponding controller (e.g., DonationController, InspectionController) receives the request, validates the JWT and role claim, then delegates to the service layer.
11. 4. The service layer executes business logic, applying the lifecycle FSM guard where relevant, and queries PostgreSQL via the Repository Pattern.
12. 5. The JSON response is returned to the client within the same HTTP transaction, within a target latency of 2 seconds.

Use cases covered: Submit/Edit/Cancel Donation (UC-2), View Available Donations (UC-5), Accept/Reject Donation (UC-6), Accept Pickup Task (UC-9), Confirm Delivery (UC-12), View Reports (UC-21).

6.2 Event-Driven Control (Asynchronous)

Lifecycle events, notification dispatch, and issue escalation use an asynchronous event-driven mechanism to avoid blocking request threads:

13. 1. A lifecycle event fires (e.g., DonationAccepted, InspectionFlagged, DeliveryConfirmed).
14. 2. The Business Logic Layer publishes an Event object to the NotificationService singleton.
15. 3. NotificationService.dispatch() identifies recipients via RBAC lookup and enqueues a Notification in the Redis priority queue.
16. 4. The Notification Server consumes the queue and dispatches via WebSocket (in-app) and/or Email/SMS gateway simultaneously, within 3 seconds of the original event.
17. 5. If delivery fails within 3 seconds, the notification enters the Redis retry queue with exponential back-off (1 s, 2 s, 4 s); failure after 3 retries is logged as a system alert for Admin review.

Use cases covered: Real-Time Notification System (UC-19), Track Donation Lifecycle (UC-20), Flag Unsafe Food (UC-14), Report Pickup Issues (UC-10).

6.3 Batch Control (Scheduled)

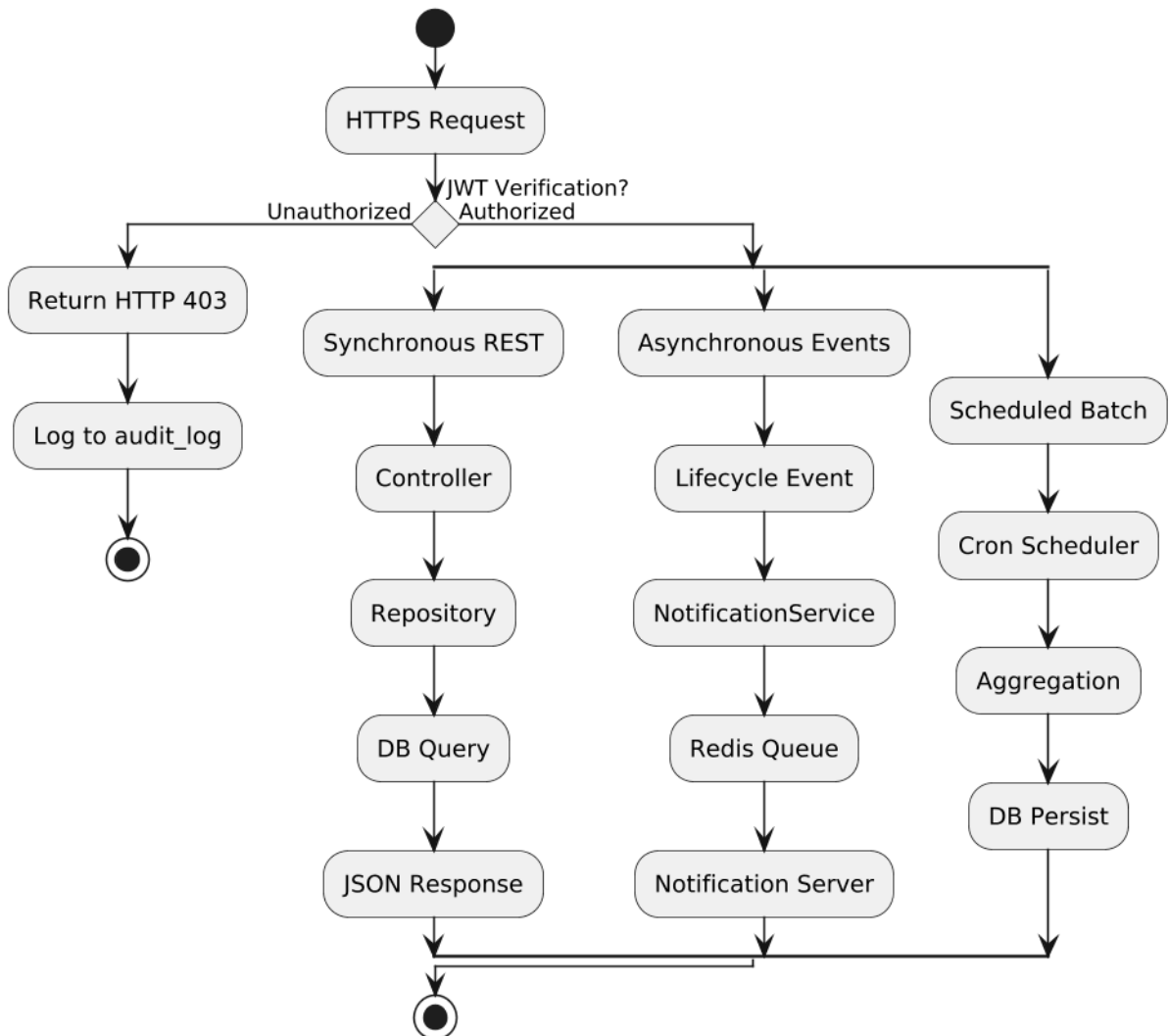
Audit archival, report generation, and lifecycle monitoring are computed on a scheduled basis:

- AdminController.generateReports() runs nightly at 01:00 IST — aggregates donation, delivery, and safety statistics for the reporting dashboard.
- NotificationService.retryFailedNotifications() runs every 5 minutes — re-attempts notifications that failed their exponential back-off retries.
- AuditLog archival runs every Sunday at 03:00 IST — archives weekly audit log snapshots to cold (S3) storage.
- Donation expiry check runs hourly — scans food_donations for items past expiryDate in SUBMITTED or INSPECTED state and flags them for Admin review.

Use cases covered: Generate Reports (UC-21), Monitor System Activity (UC-22), Track Donation Lifecycle (UC-20).

6.4 Control Flow Summary Diagram

Figure 9: Control Flow Mechanism Overview



7 Boundary Conditions

Boundary conditions specify expected system behaviour at the edges of normal operation: startup, shutdown, and failure/exception scenarios. All conditions must be handled gracefully, ensuring data integrity, auditability, and meaningful feedback to stakeholders.

7.1 System Startup

18. 1. Database Connectivity Check — On startup, the application attempts to establish a PostgreSQL connection pool (HikariCP). If connectivity is not established within 30 seconds, startup is aborted and a FATAL error is logged; Nginx returns HTTP 503 to all clients.
19. 2. Redis Connectivity Check — The Redis client probes the cluster. If unavailable, the system starts in degraded mode: session caching and notification queuing are disabled; a WARN entry is written and the Admin is alerted.
20. 3. Schema Migration — Flyway executes any pending SQL migration scripts (e.g., V1__init.sql) before the first request is accepted; failed migrations abort startup.
21. 4. Safety Rules Cache Warm-up — Active SafetyRule records are loaded from the database into the in-memory application cache before requests are accepted.
22. 5. Admin Seed Data — If the users table is empty, a default ADMIN account (admin@fdlms.vjti.ac.in) is created with a randomly generated password printed once to the startup log and then discarded.
23. 6. Notification Server Health Check — The application probes the Notification Server health endpoint; readiness is declared only after receiving HTTP 200 OK.

7.2 System Shutdown

24. 1. Graceful HTTP Drain — Nginx stops routing new connections to the application servers; in-flight requests are given a 30-second window to complete before forceful termination.
25. 2. Async Task Completion — The asynchronous notification thread pool is shut down with awaitTermination(30, SECONDS); incomplete notification dispatches are persisted to the notifications table (status = PENDING) and retried on next startup.
26. 3. Redis BGSAVE — Redis executes BGSAVE before container stop to preserve the in-memory notification queue across rolling restarts.
27. 4. Active Task Interruption — In-flight pickup and delivery tasks are marked INTERRUPTED; the relevant NGO is notified via the notification system of the interruption.
28. 5. DB Connection Pool Close — HikariCP closes all pooled JDBC connections cleanly, ensuring no dangling transactions remain on the PostgreSQL server.

7.3 Failure / Exception Conditions

Scenario	Detection Mechanism	System Response
Invalid lifecycle transition	InvalidTransitionException thrown by FoodDonation.transition()	HTTP 400 returned; transition attempt logged to lifecycle_log with REJECTED status; actor notified.
Inspection not completed before NGO acceptance	InspectionController guard check fails	HTTP 422 returned; acceptance blocked; NGO informed of inspection requirement; donation remains in SUBMITTED state.
No inspector available for assignment	NGO attempts assignment; no INSPECTOR with capacity	HTTP 409 returned; NGO alerted; donation remains in SUBMITTED state; Admin notified.

Concurrent NGO acceptance of same donation	Optimistic lock (version) conflict on food_donations	HTTP 409 Conflict returned; second NGO's request rejected; both NGOs' donation lists refreshed.
Volunteer accepts an already-taken pickup task	SELECT FOR UPDATE finds task already ASSIGNED	HTTP 409 Conflict returned; volunteer's task list refreshed automatically.
Network failure during delivery status update	DeliveryTask.updateStatus() connectivity exception	Update queued locally on device; synced to server on reconnect; no data loss.
DB transaction failure mid-lifecycle	PostgreSQL rollback triggered	Full transaction rollback; donation state unchanged; error logged to audit_log; Admin alerted.
Notification delivery failure (max retries)	NotificationService retry queue exhausted (3 attempts)	Failure logged to notifications table (status = FAILED); Admin alerted via system alert; manual resend option provided.
Donation record not found for inspection report	InspectionController lookup returns null	HTTP 404 returned; audit_log entry created; Inspector informed.
Expired JWT session	JWT filter detects token expiry	HTTP 401 returned with TOKEN_EXPIRED code; client redirected to login page.
Unauthorised access attempt	RBAC check fails for role/endpoint pair	HTTP 403 returned; attempt logged with actor ID, role, endpoint, and timestamp to audit_log.
Database connection lost mid-request	HikariCP connection timeout exceeds 30 s	HTTP 503 returned; health-check endpoint reports DB_DOWN; operations team alerted via monitoring system.

7.4 Input Boundary Validation

- Food quantity: must be a positive float greater than 0.1 kg and not exceed 10,000 kg per donation; rejected with HTTP 400 otherwise.
- Expiry date: must be $\geq$ today's date + 24 hours; donations expiring within 24 hours are flagged as URGENT but not blocked.
- Pickup location: latitude and longitude must be valid WGS-84 coordinates; address string must not be empty.
- Lifecycle state transitions: validated against the defined FSM allowed-transition table; illegal transitions return HTTP 422 with the current and attempted states in the error body.
- Inspection outcome: must be one of {SAFE, UNSAFE, REQUIRES_REVIEW}; unknown values rejected with HTTP 400.
- Role assignment by Admin: restricted to the enumerated set {DONOR, NGO, VOLUNTEER, DELIVERY_PARTNER, INSPECTOR, ADMIN}; unknown role strings rejected with HTTP 400.
- Pickup task status updates: must follow ASSIGNED $\rightarrow$ EN_ROUTE $\rightarrow$ ARRIVED $\rightarrow$ PICKED_UP; out-of-sequence updates rejected with HTTP 422.
- Delivery task status updates: must follow ACCEPTED $\rightarrow$ IN_TRANSIT $\rightarrow$ (DELAYED $\rightarrow$)* DELIVERED; out-of-sequence updates rejected with HTTP 422.
- Penalty severity: must be one of {MINOR, MODERATE, SEVERE}; server-side computed; client-submitted values are ignored.

Conclusion

This System Design Document for the Food Donation and Lifecycle Management System (FDLMS) has been prepared by Rudrakshi Chincholkar (241071014) and Nihira Neralwar (241071049) for VJTI Mumbai, Academic Year 2025–26, as part of SE Laboratory Experiment 8A.

The document addresses all seven required activities:

29. 1. Design Goals — 21 measurable goals across performance, reliability, security, scalability, usability, structured by priority (Critical / High / Medium) with quantified metrics.
30. 2. Subsystem Decomposition — Six cohesive subsystems (Auth, Donation, Inspection, Logistics, Notification, Admin) with per-subsystem EBC class descriptions and a consolidated full-system class diagram.
31. 3. Hardware / Software Mapping — Three-tier + notification-tier deployment architecture with UML deployment diagram covering seven hardware nodes and all communication protocols.
32. 4. Data Management — PostgreSQL 16 schema mapping for all entity classes, a five-layer Redis caching strategy, ACID transaction policy, optimistic locking, immutable audit tables, and a 7-year retention policy.
33. 5. Access Control — RBAC with six roles, an 18-operation permission matrix, RS256 JWT authentication, and a 7-step authentication flow with account lockout policy.
34. 6. Control Flow — Synchronous REST, asynchronous event-driven notification dispatch, and scheduled batch mechanisms with a control-flow summary diagram.
35. 7. Boundary Conditions — Startup (6 checks), shutdown (5 steps), 12 exception scenarios with detection mechanisms and system responses, and 9 input boundary validation rules.

All class names, controller names, use-case identifiers, and actor roles are fully consistent with the Problem Statement (Experiment 3A), EBC Class Diagrams (Experiment 4B), and the Requirements Analysis Document. The design enforces strict lifecycle integrity through a Finite State Machine, ensures food safety through mandatory pre-acceptance inspection, and provides end-to-end auditability through immutable, timestamped logs.